

Cài đặt, gỡ lỗi và kiểm thử (Implementation, Debugging, Testing)

PHỤ THUỘC A/04 cho bất biến — công cụ phòng lỗi mạnh nhất; C/13 cho lời giải vét cạn, thứ đóng vai trò trọng tài trong stress test.

TÓM TẮT

Chương này nói về khoảng cách giữa “tôi biết phải làm gì” và “chương trình chạy đúng”. Ba phần: viết code sao cho ít lỗi ngay từ đầu — chủ yếu bằng cách viết bất biến trước khi viết vòng lặp; danh mục các lỗi kinh điển kèm cách phòng từng loại; và quan trọng nhất, một quy trình gỡ lỗi có hệ thống thay cho việc sửa mò. Trung tâm của quy trình đó là *stress test*: sinh dữ liệu nhỏ ngẫu nhiên, chạy song song lời giải nhanh và lời giải vét cạn, so kết quả, rồi thu nhỏ phản ví dụ. Nếu chỉ nhớ một điều: khi bạn không biết vì sao code sai, đừng đọc lại code — hãy để máy tìm cho bạn một phản ví dụ nhỏ, vì mười phút viết bộ sinh test gần như luôn rẻ hơn một giờ nhìn chằm chằm màn hình.

KIẾN THỨC CẦN CÓ TRƯỚC

- **stress test** — chạy hai lời giải — một nhanh, một chậm nhưng chắc đúng — trên hàng nghìn dữ liệu nhỏ sinh ngẫu nhiên, và dừng lại ở lần đầu hai kết quả khác nhau.
- **thu nhỏ phản ví dụ** — từ một dữ liệu làm chương trình sai, tìm dữ liệu nhỏ hơn vẫn giữ được lỗi — vì lỗi trên dữ liệu ba phần tử dễ hiểu hơn nhiều so với lỗi trên ba nghìn phần tử.
- **khẳng định (assertion)** — câu lệnh kiểm tra một điều kiện phải đúng tại một điểm trong chương trình; nếu sai thì dừng ngay tại chỗ phát sinh thay vì để lỗi lan ra.
- **lỗi im lặng** — lỗi không làm chương trình sập mà chỉ làm kết quả sai — loại nguy hiểm nhất, vì nó chỉ lộ ra khi có người đối chiếu.
- **dữ liệu biên** — các trường hợp ở ranh giới: rỗng, một phần tử, mọi phần tử bằng nhau, giá trị lớn nhất, giá trị âm.
- **tái lập** — khả năng chạy lại và gặp đúng lỗi cũ; với chương trình có ngẫu nhiên, điều này đòi phải ghi lại hạt giống.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao “đọc lại code” là chiến lược gỡ lỗi kém, dù đó là việc ai cũng làm đầu tiên?
- Q2. Stress test cần một lời giải vét cạn — nghe như làm gấp đôi công việc. Vì sao nó vẫn tiết kiệm thời gian?
- Q3. Vì sao lỗi trên dữ liệu nhỏ lại dễ sửa hơn lỗi trên dữ liệu lớn tới mức đáng bỏ công thu nhỏ phản ví dụ?
- Q4. Loại lỗi nào sống sót qua mọi bộ test ngẫu nhiên, và phải phòng bằng cách khác?
- Q5. Khi chương trình có yếu tố ngẫu nhiên, làm sao gỡ lỗi được?

Có một cảnh quen thuộc: bạn nộp bài, nhận về “sai ở test 12”, và bắt đầu đọc lại code. Đọc ba lần, mọi dòng đều hợp lý. Bạn đổi một dấu < thành <=, nộp lại, vẫn sai. Đổi lại như cũ, thêm một trường hợp riêng, vẫn sai. Nửa giờ trôi qua và bạn không biết mình đang tiến hay lùi.

Vấn đề của cảnh này không phải kỹ năng lập trình mà là *thiếu quy trình*. Đọc lại code là cách gỡ lỗi kém vì hai lý do: bạn đọc lại chính suy nghĩ của mình, nên bạn sẽ đọc thấy điều bạn *định* viết chứ không phải điều bạn *đã* viết; và bạn không có thông tin mới nào — đúng tình trạng “lặp lại một biểu diễn” mà chương A/01 đã mô tả.

Chương này thay cảnh đó bằng một quy trình. Quy trình ấy dựa trên một nguyên tắc duy nhất: *hãy để máy tìm cho bạn một ví dụ sai nhỏ, rồi mới dùng đầu.*

1.1 Phòng bệnh: viết code ít lỗi ngay từ đầu

Cách rẻ nhất để sửa lỗi là không tạo ra nó. Bốn thói quen có hiệu quả rõ rệt.

Viết bất biến trước khi viết vòng lặp. Đây là lời khuyên của chương A/04 và nó thuộc về chương này nhiều nhất. Một dòng chú thích — “sau vòng lặp thứ i , $best$ là đáp án của $a[0..i]$ ” — biến mọi quyết định về chỉ số từ đoán mò thành suy diễn. Phần lớn lỗi lệch một đơn vị chết ngay tại đây.

Chọn quy ước và giữ nhất quán. Đánh chỉ số từ 0 hay từ 1, đoạn đóng $[l, r]$ hay nửa mở $[l, r)$ — chọn một và dùng cho toàn bài. Phần lớn lỗi biên đến từ việc trộn hai quy ước trong cùng một chương trình. Nửa mở là quy ước ít gây lỗi hơn cho các bài chia đoạn, vì độ dài bằng đúng $r - l$ và phép chia đôi không có trường hợp riêng.

Ưu tiên rõ ràng hơn ngắn. Một dòng code khéo léo tiết kiệm ba dòng nhưng tốn hai mươi phút khi phải gỡ lỗi. Trong thi đấu điều này còn quan trọng hơn ngoài đời, vì bạn không có thời gian đọc lại thứ mình không hiểu ngay.

Đặt khẳng định ở những chỗ bạn tin chắc. Nghịch lý là ở chỗ: nơi đáng đặt khẳng định nhất chính là nơi bạn *chắc chắn* nó luôn đúng, vì đó là những giả định ngầm mà bạn sẽ không bao giờ nghĩ tới khi gỡ lỗi. Khẳng định làm lỗi dừng lại tại nơi phát sinh thay vì để nó lan ra và chỉ lộ ở kết quả cuối.

1.2 Danh mục lỗi kinh điển

Phần lớn lỗi trong lập trình thuật toán rơi vào một số ít loại. Thuộc danh mục này giúp bạn kiểm nhanh trước khi nộp, và cột thứ ba là cách phòng chứ không phải cách sửa.

Loại lỗi	Dấu hiệu	Cách phòng
Tràn số	Kết quả âm bất ngờ, hoặc sai chỉ với dữ liệu lớn	Ước lượng giá trị lớn nhất <i>trước</i> ; mặc định dùng 64 bit cho tổng và tích
Lệch một đơn vị	Sai đúng ở phần tử đầu hoặc cuối	Viết bất biến; chọn một quy ước đoạn và giữ nhất quán
Chưa khởi tạo	Kết quả đổi giữa các lần chạy	Khởi tạo tường minh; đặt khẳng định về miền giá trị hợp lệ
Hàm so sánh không nhất quán	Sắp xếp cho kết quả lạ, đôi khi sập	Kiểm quan hệ có bắc cầu; không dùng “gần bằng” làm khoá sắp xếp
Đệ quy quá sâu	Sập với dữ liệu lớn, chạy tốt với dữ liệu nhỏ	Ước lượng độ sâu tối đa; chuyển sang bản lặp khi cần
Chia số nguyên	Sai với số âm hoặc mất phần lẻ	Nhân chéo thay vì chia; cẩn thận với hướng làm tròn của số âm
Phần dư âm	Kết quả âm sau phép trừ modulo	Cộng thêm m rồi lấy dư lần nữa $(D/21)$
Trường hợp $n = 0$ hoặc $n = 1$	Sai đúng ở test đầu hoặc test biên	Luôn thử tay hai ca này trước khi nộp
Đọc dữ liệu chậm	Quá thời gian dù thuật toán đúng bậc	Dùng cách đọc nhanh khi n tới hàng triệu

Có một lỗi thứ mười không nằm trong bảng vì nó thuộc loại khác: *hiểu sai đề*. Nó chiếm tỉ lệ cao hơn nhiều người thừa nhận, và cách phòng duy nhất là bước 1 của Pólya — phát biểu lại đề bằng lời của mình và tính tay ví dụ trong đề (A/01).

1.3 Stress test: kỹ thuật đánh giá nhất chương

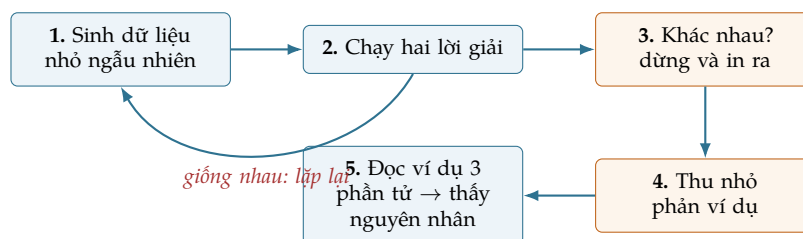
Đây là quy trình thay thế cho việc sửa mò, và nó gồm bốn phần.

Một — bộ sinh dữ liệu. Một chương trình nhỏ sinh dữ liệu ngẫu nhiên *hợp lệ* với kích thước *rất nhỏ*: $n \leq 8$, giá trị trong $[-3, 3]$. Kích thước nhỏ là điểm mấu chốt, vì nó vừa làm lỗi dễ hiểu vừa làm các trường hợp trùng lặp và biên xuất hiện thường xuyên thay vì hiếm.

Hai — lời giải trọng tài. Một lời giải vét cạn, chậm nhưng bạn tin chắc là đúng. Đây chính là lý do chương C/13 nói “luôn viết vét cạn trước”: nó không chỉ giúp bạn nghĩ, nó còn là công cụ kiểm thử.

Ba — vòng lặp so sánh. Chạy cả hai trên cùng dữ liệu, so kết quả, dừng lại ở lần đầu khác nhau và in dữ liệu đó ra. Vài dòng kịch bản, chạy vài nghìn ca trong vài giây.

Bốn — thu nhỏ. Khi đã có một dữ liệu sai, hãy tìm dữ liệu *nhỏ hơn* vẫn sai: thử bỏ từng phần tử, giảm từng giá trị, và giữ lại thay đổi nào vẫn giữ được lỗi. Việc này tự động hoá được bằng một vòng lặp đơn giản, và nó biến một phần ví dụ 8 phần tử thành 3 phần tử — ở kích thước đó bạn thường nhìn ra nguyên nhân ngay lập tức.



Hình 1.1. Vòng stress test. Điểm quan trọng nhất là bước 1 phải sinh dữ liệu *rất nhỏ* — điều này vừa làm phần ví dụ dễ đọc vừa làm các trường hợp biên xuất hiện liên tục thay vì hiếm. Bước 4 thường bị bỏ, nhưng nó là bước biến một phần ví dụ khó hiểu thành một phần ví dụ hiển nhiên.

Stress test không chỉ dùng để gỡ lỗi. Nó còn là cách kiểm chứng một *ý tưởng* trước khi tin vào nó: khi bạn nghĩ ra một tiêu chí tham lam (C/15) hoặc một công thức quy hoạch động, năm phút stress test cho câu trả lời chắc chắn hơn nhiều so với mười phút cố chứng minh trong đầu.

CẦN NHỚ

Khi bí trong lúc gỡ lỗi, thứ tự nên làm là: (i) kiểm dữ liệu biên bằng tay — $n = 0, 1$, mọi phần tử bằng nhau; (ii) viết stress test; (iii) thu nhỏ phản ví dụ; (iv) chỉ khi đó mới đọc code. Đảo ngược thứ tự này — đọc code trước — là cách phần lớn thời gian gỡ lỗi bị lãng phí.

1.4 Khi không có lời giải trọng tài

Đôi khi bài quá khó để viết vết cạn, hoặc kết quả không so sánh trực tiếp được (có nhiều đáp án đúng). Vẫn còn ba cách.

Kiểm tính chất thay vì kiểm giá trị. Nếu bài yêu cầu một cách sắp xếp thoả điều kiện, hãy viết một hàm *kiểm tra* kết quả có hợp lệ không thay vì so với đáp án. Đây là ý tưởng nền của kiểm thử dựa trên tính chất, và nó thường dễ hơn viết vết cạn nhiều.

So hai cách cài đặt khác nhau. Nếu bạn có hai thuật toán cho cùng bài — ví dụ một bản đệ quy và một bản lặp, hoặc một bản dùng cây phân đoạn và một bản dùng phân rã căn — so chúng với nhau. Hai bản có thể cùng sai vì dùng chung mô hình hoặc cùng một giả thiết. So chéo là bằng chứng bổ sung, không thay thế trọng tài độc lập và các bất biến.

Kiểm bất biến trong lúc chạy. Rải khắp định kiểm các bất biến ở những điểm then chốt, rồi chạy trên dữ liệu lớn. Cách này không cần trọng tài và bắt được lỗi ngay tại nơi cấu trúc dữ liệu bị hỏng.

1.5 Gỡ lỗi như một bài toán tìm kiếm

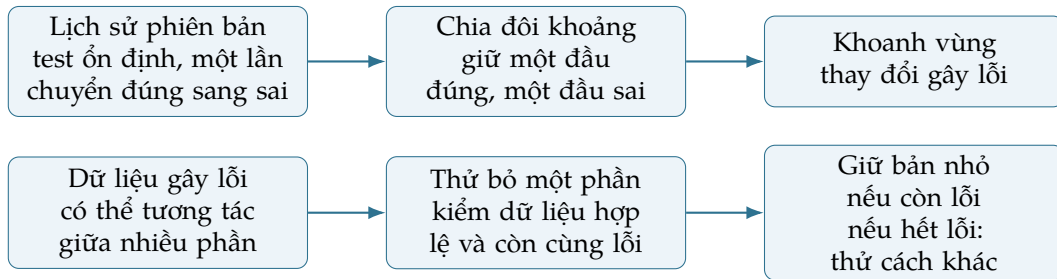
Khi đã có phản ví dụ nhỏ mà vẫn chưa thấy nguyên nhân, hãy coi việc gỡ lỗi là bài toán tìm kiếm — chọn phép thu hẹp theo cấu trúc của lỗi, thay vì mặc định mọi thứ đều đơn điệu.

Trên trục *không gian*: đặt các điểm kiểm tra ở giữa chương trình, xác nhận trạng thái tại đó đúng hay sai, rồi loại một nửa. Chỉ loại được một nửa nếu phép kiểm bảo đảm trạng thái lỗi không bị che hoặc sửa ở bước sau; nói chung trạng thái đúng/sai có thể xen kẽ.

Trên trục *thời gian*: nếu code trước đây chạy đúng, hãy tìm thay đổi đầu tiên làm hỏng bằng cách chia đôi lịch sử — đây chính là công cụ *git bisect*, và điều kiện dùng được của nó là tính đơn điệu, đúng như mọi tìm kiếm nhị phân khác.

Trên trục *dữ liệu*: đó chính là bước thu nhỏ phản ví dụ ở mục 3.

Hình 1.2 phân biệt phép chia đôi có bất biến với việc thu nhỏ dữ liệu phải kiểm lại sau mỗi lần thử.



Hình 1.2. Hai quy trình thu hẹp khác nhau. Trên: tìm ranh giới trong một lịch sử có giả thiết đơn điệu. Dưới: thu nhỏ phân ví dụ; không suy ra rằng một nửa đúng thì nửa còn lại chắc chắn chứa toàn bộ nguyên nhân.

Ba trục này dùng chung ý tưởng thu hẹp, nhưng không có cùng bảo đảm thuật toán, và nhận ra điều đó giúp bạn luôn có một nước đi tiếp theo thay vì bế tắc.

1.6 Ba việc riêng cho chương trình có ngẫu nhiên

Câu hỏi mở đầu thứ năm hỏi về việc gỡ lỗi khi chương trình có yếu tố ngẫu nhiên, và câu trả lời gồm ba việc rất cụ thể.

Ghi lại hạt giống. Mỗi lần chạy, in ra hạt giống đã dùng. Khi gặp lỗi, chạy lại với đúng hạt giống đó và bạn tái lập được. Không có bước này thì mọi lỗi phụ thuộc ngẫu nhiên đều gần như không sửa được.

Tách phần ngẫu nhiên khỏi phần logic. Nếu nguồn ngẫu nhiên được truyền vào như một tham số thay vì gọi trực tiếp trong hàm, bạn thay được nó bằng một dãy cố định khi kiểm thử.

Kiểm bằng thống kê, không bằng một lần chạy. Với thuật toán ngẫu nhiên, một lần chạy đúng không nói gì. Hãy chạy hàng nghìn lần và kiểm phân phối kết quả — ví dụ với một hàm trộn ngẫu nhiên, đếm tần suất mỗi hoán vị trên $n = 4$ và kiểm chúng gần đều nhau.

1.7 Biến mỗi lỗi thành một phép kiểm có thể chạy lại

Một lần sửa chỉ có giá trị lâu dài nếu phần ví dụ trở thành ca kiểm thử hồi quy. Bảng 1.1 đưa ra ba lớp kiểm bổ sung nhau. Với một bài trả về nhiều lời giải hợp lệ, hãy so giá trị mục tiêu và dùng bộ kiểm chứng tính hợp lệ, không so nguyên văn hai danh sách kết quả.

Bảng 1.1. Ba lớp kiểm thử và giới hạn của bằng chứng chúng cung cấp.

Lớp kiểm	Ví dụ	Bắt được	Chưa bảo đảm
Ca biên có chủ đích	Mảng rỗng, toàn âm, giá trị trùng, không có nghiệm	Sai khởi tạo và quy ước biên	Phối hợp bất ngờ giữa nhiều điều kiện
So với vết cặn	Duyệt mọi tập con ở $n \leq 12$ rồi so giá trị	Sai logic của thuật toán nhanh	Lỗi chỉ xuất hiện khi dữ liệu lớn
Quan hệ biến đổi	Sắp lại đầu vào không đổi MST; đảo đoạn không đổi tổng	Sai phụ thuộc thứ tự ngoài đặc tả	Đúng cho mọi dữ liệu

Khi test sai, lưu cả đầu vào cụ thể, kết quả mong đợi, kết quả thực tế và seed; seed một mình chưa đủ nếu generator hay phiên bản thư viện đã đổi. Thu nhỏ nhưng giữ đúng đặc tả: bỏ một cạnh có thể làm đồ thị không còn liên thông, khiến test không còn thuộc miền bài toán. Sau khi

sửa, chạy lại ca gốc, ca đã thu nhỏ và bộ đối chiếu; đừng chỉ chạy đúng ví dụ nhỏ vừa dùng để sửa.

1.8 Gốc rễ: từ khẳng định của Turing tới thu nhỏ tự động

Ý tưởng đặt khẳng định vào chương trình để kiểm tra trạng thái xuất hiện rất sớm — Turing đã dùng nó năm 1949 trong bài viết về kiểm tra một chương trình lớn, như chương A/04 đã kể. Nhưng phải mất nhiều thập kỷ để nó thành thói quen phổ biến, và lý do là bối cảnh: khi máy tính còn đắt, việc để chương trình tự dừng vì một khẳng định bị coi là lãng phí thời gian máy.

Câu nói của Dijkstra — kiểm thử cho thấy sự hiện diện của lỗi, không cho thấy sự vắng mặt — thường bị dùng như một lời chê kiểm thử, nhưng ý ông tinh tế hơn: kiểm thử là một công cụ *phát hiện* rất tốt và một công cụ *bảo chứng* rất tệ, nên đừng nhầm hai vai trò. Toàn bộ mục 3 của chương này khai thác đúng vai trò thứ nhất.

Ba kỹ thuật hiện đại đáng nhắc vì chúng đều là hình thức hoá của những việc mà người giải bài vẫn làm bằng tay. *Kiểm thử ngẫu nhiên có hệ thống* bắt đầu được nghiên cứu nghiêm túc từ đầu thập niên 1990, khi người ta phát hiện rằng ném dữ liệu ngẫu nhiên vào các tiện ích hệ thống làm sập một tỉ lệ đáng ngạc nhiên trong số chúng — một kết quả gây sốc vì các phần mềm đó đã được dùng hàng ngày trong nhiều năm. *Kiểm thử dựa trên tính chất* ra đời cuối thập niên 1990 với ý tưởng: thay vì viết từng ca kiểm thử, hãy phát biểu một tính chất phải luôn đúng và để máy sinh dữ liệu tìm phản ví dụ — đúng mục 4 ở trên. Và *thu nhỏ tự động* được hình thức hoá cuối thập niên 1990 dưới tên gỡ lỗi delta: một quy trình thử các tập con và phần bù để tìm phản ví dụ 1-tối-tiểu (bỏ thêm một đơn vị thì không còn tái hiện lỗi), không bảo đảm nhỏ nhất toàn cục^[1], tức là bước 4 ở mục 3 nhưng có phân tích chi phí đàng hoàng.

Điểm chung của ba kỹ thuật: chúng đều chuyển công việc từ người sang máy, và cả ba đều dựa trên cùng một quan sát — *máy tìm phản ví dụ giỏi hơn người, còn người hiểu phản ví dụ giỏi hơn máy*. Quy trình ở mục 3 chỉ là cách phân công đúng theo quan sát đó.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1949 — Turing	Kiểm tra một chương trình dài mà chạy thử không xuể.	Khẳng định tại từng điểm; ý tưởng có từ rất sớm nhưng chỉ thành thói quen phổ biến khi thời gian máy trở nên rẻ.
Thập niên 1970 — Dijkstra	“Khủng hoảng phần mềm”: lỗi vượt khả năng kiểm soát bằng chạy thử.	Kiểm thử là công cụ phát hiện lỗi, không phải bảo chứng — phân vai đúng thay vì bỏ kiểm thử.
Đầu thập niên 1990 — kiểm thử bằng dữ liệu ngẫu nhiên	Ném dữ liệu ngẫu nhiên vào các tiện ích hệ thống làm sập nhiều bất ngờ.	Chứng minh rằng phần mềm dùng hàng ngày vẫn đầy lỗi ở dữ liệu bất thường; sinh ra cả một nhánh kiểm thử tự động.
Cuối thập niên 1990 — gỡ lỗi delta	Phản ví dụ tìm được thường quá lớn để hiểu.	Thuật toán thử tập con và phần bù để thu nhỏ dữ liệu — chính là bước 4 của stress test, có phân tích đàng hoàng.
Khoảng 2000 — kiểm thử dựa trên tính chất	Viết từng ca kiểm thử vừa tốn công vừa bỏ sót.	Phát biểu tính chất phải luôn đúng, để máy sinh dữ liệu tìm phản ví dụ; cách làm khi không có lời giải trọng tài.

1.9 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Kiểm chứng phần cứng	Mô phỏng ngẫu nhiên có ràng buộc, kiểm chứng hình thức	Cùng cấu trúc với stress test nhưng ở quy mô công nghiệp: sinh kích thích ngẫu nhiên hợp lệ và so với một mô hình tham chiếu — chính là “lời giải trọng tài” của mục 3.
Hàng không	Bảng kiểm trước khi cất cánh	Danh mục lỗi ở mục 2 chính là một bảng kiểm. Bài học từ ngành hàng không: bảng kiểm hiệu quả không phải vì phi công không biết, mà vì nó chống lại việc <i>bỏ sót dưới áp lực</i> — đúng tình huống trong kỳ thi.
Y khoa	Chẩn đoán phân biệt, xét nghiệm loại trừ	Gỡ lỗi bằng cách chia đôi vùng nghi ngờ giống hệt việc loại trừ dần các chẩn đoán. Cùng nguyên tắc: chọn phép thử nào loại được nhiều khả năng nhất, không phải phép thử dễ nhất.
Khoa học thực nghiệm	Tái lập kết quả, ghi sổ thí nghiệm	Việc ghi hạt giống ngẫu nhiên ở mục 6 chính là yêu cầu tái lập của khoa học. Một kết quả không tái lập được thì không kiểm chứng được — đúng với thí nghiệm và với lỗi phần mềm.
Ước lượng trạng thái, SLAM	Chạy trên đoạn dữ liệu ngắn, kiểm bất biến của bộ lọc	Áp dụng trực tiếp mục 3 và 4: dựng “ví dụ nhỏ nhất” là một đoạn vài giây thay vì cả chuỗi, và kiểm các bất biến (hiệp phương sai xác định dương, hướng trọng lực không trôi) bằng khẳng định trong lúc chạy.
Thống kê	Kiểm định giả thuyết	Việc kiểm thuật toán ngẫu nhiên bằng phân phối kết quả (mục 6) là một bài kiểm định thống kê. Cạm bẫy cũng giống: một lần quan sát không nói gì, và cần đủ số lần lặp để kết luận có nghĩa.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai lý do. Thứ nhất, bạn đọc lại chính suy nghĩ của mình nên bạn thấy điều bạn *định* viết chứ không phải điều bạn *đã* viết — đây là hiện tượng tâm lý rất mạnh và không khắc phục được bằng cách cố gắng hơn. Thứ hai, đọc lại không tạo ra thông tin mới; bạn đang lặp lại cùng một biểu diễn, đúng trạng thái bí mà A/01 mô tả. Stress test thì tạo ra thông tin mới: một dữ liệu cụ thể mà chương trình sai, và đó là thứ thu hẹp không gian nguyên nhân.
- A2. Vì lời giải vét cạn thường chỉ tốn năm tới mười phút — nó không cần nhanh, không cần đẹp, chỉ cần đúng — trong khi việc sửa mò một lỗi không hiểu có thể tốn hàng giờ và không có gì bảo đảm kết thúc. Ngoài ra bạn thường đã viết vét cạn rồi, vì C/13 khuyên viết nó như bước đầu của quá trình nghĩ. Và nó còn dùng lại cho các bài sau: một bộ sinh test và một khung stress test viết một lần dùng mãi.
- A3. Vì số lượng thứ có thể sai tỉ lệ với kích thước dữ liệu. Trên mảng ba phần tử, bạn theo dõi được toàn bộ trạng thái bằng mắt và thường nhìn ra nguyên nhân trong một phút. Trên mảng ba nghìn phần tử, bạn không biết bắt đầu từ đâu, và các lỗi khác nhau trông giống hệt nhau ở đầu ra. Việc thu nhỏ tự động hoá được bằng một vòng lặp thử bỏ từng phần tử, nên chi phí gần như bằng không so với lợi ích.
- A4. Lỗi ở trường hợp *biên* và trường hợp *cấu trúc đặc biệt* — mảng rỗng, một phần tử, mọi phần tử bằng nhau, đồ thị không liên thông, giá trị lớn nhất gây tràn. Bộ sinh ngẫu nhiên thông thường hầu như không tạo ra chúng. Cách phòng có hai phần: sinh dữ liệu với *miền giá trị rất nhỏ* để ép trùng lặp và biên xuất hiện thường xuyên; và bổ sung một danh sách ca biên viết tay chạy trước mọi lần nộp.

- A5.** Ba việc. Ghi lại hạt giống mỗi lần chạy và in ra, để tái lập được đúng lần chạy hỏng. Truyền nguồn ngẫu nhiên vào như tham số thay vì gọi trực tiếp, để thay được bằng dãy cố định khi kiểm thử. Và kiểm bằng thống kê thay vì bằng một lần chạy — chạy hàng nghìn lần rồi kiểm phân phối, vì với thuật toán ngẫu nhiên thì một lần đúng không chứng minh gì.

NÓI LẠI THẬT ĐƠN GIẢN

Khi chương trình sai mà bạn không biết vì sao, phản xạ tự nhiên là đọc lại code. Đó gần như luôn là cách chậm nhất, vì bạn sẽ đọc thấy cái mình *định* viết chứ không phải cái mình *đã* viết — giống như tự dò lỗi chính tả trong bài văn của chính mình.

Cách nhanh hơn nhiều là bắt máy tìm lỗi. Bạn viết thêm hai chương trình rất ngắn: một cái sinh ra dữ liệu nhỏ ngẫu nhiên, và một cái giải bài theo cách ngu ngốc nhất nhưng chắc chắn đúng — thử hết mọi khả năng. Rồi cho cả hai chạy hàng nghìn lần và so đáp án. Khi chúng khác nhau, bạn có trong tay một ví dụ cụ thể mà chương trình sai.

Còn một bước nữa mà nhiều người bỏ, và nó là bước đáng giá nhất: hãy làm cho ví dụ đó *nhỏ hết mức*. Thử bỏ bớt một phần tử, nếu vẫn sai thì bỏ tiếp. Một ví dụ sai với ba phần tử thường tự nói ra nguyên nhân; cùng cái lỗi ấy trên ba nghìn phần tử thì không nói gì cả.

Cả quy trình này chỉ tốn mười phút để viết lần đầu, và bạn dùng lại nó cho mọi bài sau. So với việc ngồi sửa mò nửa tiếng và không chắc đã xong, đó là một trong những đánh đổi tốt nhất trong nghề.

Cuối cùng, gỡ lỗi là tìm kiếm bằng các phép kiểm có thể lặp lại. Dùng chia đôi khi có ranh giới đúng/sai; với dữ liệu, thử thu nhỏ và chỉ giữ thay đổi còn tái hiện cùng lỗi. Một phản ví dụ không thu nhỏ thêm được theo các phép thử hiện có chưa chắc nhỏ nhất toàn cục.

Một câu: *Đừng đọc lại code khi bí — hãy để máy tìm cho bạn một ví dụ sai nhỏ.*

Chỗ không rút gọn được: Lỗi phụ thuộc thời gian hoặc phụ thuộc môi trường (nhiều luồng, phân cứng) không tái lập được bằng stress test thông thường; đó là một lớp khó riêng.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vì sao không nên đọc lại code trước	Bạn đọc thấy điều mình định viết, và không tạo ra thông tin mới. Stress test tạo ra thông tin mới: một dữ liệu cụ thể sai.
••• Bốn bước stress test	Sinh dữ liệu <i>rất nhỏ</i> → chạy lời giải nhanh và vét cạn → dừng ở lần khác nhau đầu tiên → thu nhỏ phản ví dụ.
••• Vì sao dữ liệu phải nhỏ	Phản ví dụ dễ đọc, và các trường hợp trùng lặp/biên xuất hiện thường xuyên thay vì hiếm. Miền giá trị nhỏ cũng quan trọng như n nhỏ.
••• Thứ tự khi gỡ lỗi	Kiểm ca biên bằng tay → stress test → thu nhỏ → mới đọc code. Đảo ngược thứ tự này là cách lãng phí thời gian phổ biến nhất.
••• Gỡ lỗi là bài toán tìm kiếm	Chia đôi trên ba trục: không gian (điểm kiểm tra giữa chương trình), thời gian (git bisect), dữ liệu (thu nhỏ phản ví dụ).
••• Danh mục lỗi kinh điển	Tràn số; lệch một đơn vị; chưa khởi tạo; hàm so sánh không nhất quán; đệ quy quá sâu; chia số nguyên; phần dư âm; $n = 0, 1$; đọc dữ liệu chậm.
•• Khi không có lời giải trọng tài	Kiểm <i>tính chất</i> thay vì giá trị; so hai cách cài đặt khác nhau; rải khẳng định kiểm bất biến trong lúc chạy.
•• Nơi nên đặt khẳng định	Ở những chỗ bạn <i>chắc chắn</i> luôn đúng — vì đó là các giả định ngầm mà bạn sẽ không nghĩ tới khi gỡ lỗi.
•• Ba việc với chương trình ngẫu nhiên	Ghi và in hạt giống; truyền nguồn ngẫu nhiên như tham số; kiểm bằng phân phối trên nhiều lần chạy chứ không bằng một lần.
•• Quy ước đoạn	Chọn nửa mở $[l, r)$ hoặc đóng $[l, r]$ và giữ nhất quán cả bài; nửa mở ít gây lỗi hơn vì độ dài bằng $r - l$.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Stress test không chỉ để gỡ lỗi	Còn để <i>kiểm chứng ý tưởng</i> : năm phút stress test một tiêu chí tham lam chắc chắn hơn mười phút cổ chứng minh trong đầu.
● Câu của Dijkstra, hiểu đúng	Kiểm thử là công cụ phát hiện lỗi rất tốt và công cụ bảo chứng rất tệ — đừng nhầm hai vai trò, không phải dùng kiểm thử.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Viết bộ sinh test tốn bao lâu, và có đáng trong kỳ thi không?

| Khoảng năm phút nếu bạn đã có khung sẵn, và đáng trong hầu hết tình huống. Phép so sánh đúng là: bạn đang cân nhắc giữa năm phút chắc chắn có tiến triển và một khoảng thời gian không xác định để sửa mò. Ngoại lệ hợp lý: khi bạn còn dưới mười phút và lỗi rõ ràng là lỗi cài đặt nhỏ. Lời khuyên thực dụng: chuẩn bị sẵn khung stress test trong thư viện cá nhân (A/02) để mỗi lần chỉ phải viết bộ sinh.

2. Độ quy của tôi sập với n lớn nhưng chạy tốt với n nhỏ. Sửa thế nào?

| Đó gần như chắc chắn là tràn ngăn xếp. Ba lựa chọn: tăng giới hạn ngăn xếp nếu môi trường cho phép; chuyển sang bản lập với ngăn xếp tường minh (B/07); hoặc giảm độ sâu bằng cách đổi thứ tự duyệt — ví dụ với cây, duyệt con nặng sau cùng. Cách kiểm nhanh: chạy thử với dữ liệu hình đường thẳng (cây suy biến thành chuỗi), vì đó là trường hợp độ sâu lớn nhất và bộ sinh ngẫu nhiên hầu như không tạo ra.

3. Làm sao biết lỗi là tràn số?

| Ba dấu hiệu. Kết quả âm ở nơi phải dương; kết quả đúng với dữ liệu nhỏ và sai với dữ liệu lớn; hoặc kết quả sai lệch một lượng gần bằng lũy thừa của 2. Cách kiểm nhanh nhất là đổi tạm mọi kiểu số sang 128 bit hoặc sang số nguyên lớn và chạy lại — nếu đúng thì bạn đã xác định được nguyên nhân. Cách phòng là ước lượng giá trị lớn nhất *trước khi cài*, cùng lúc với việc ước lượng độ phức tạp (A/03).

4. Tôi nên đặt bao nhiêu khẳng định?

| Đủ để mỗi cấu trúc dữ liệu quan trọng được kiểm bắt biến ít nhất một lần mỗi thao tác, nhưng không nhiều tới mức làm chậm chương trình đáng kể. Quy tắc thực dụng: đặt khẳng định ở ranh giới giữa các phần — đầu vào của một hàm, sau khi cập nhật một cấu trúc — và ở những giả định ngầm. Nhớ tắt chúng trong bản nộp nếu chúng tốn thời gian; nhưng đừng xóa, chỉ tắt.

5. Chương trình chạy đúng trên máy tôi, sai trên máy chấm. Nguyên nhân hay gặp?

| Bốn nguyên nhân theo thứ tự tần suất. Biến chưa khởi tạo — giá trị rác khác nhau giữa hai môi trường. Hành vi không xác định như tràn số có dấu hoặc truy cập ngoài mảng — có thể “chạy đúng” ở nơi này và sai ở nơi khác. Khác biệt về kích thước kiểu số hoặc về cách làm tròn số thực. Và giới hạn ngăn xếp khác nhau. Cả bốn đều bắt được bằng cách bật các tùy chọn kiểm tra của trình biên dịch khi chạy thử ở nhà.

6. Có cách nào kiểm thử một lời giải mà bài có nhiều đáp án đúng?

| Có, và đó là mục 4: viết một hàm *kiểm tra tính hợp lệ* thay vì so với đáp án mẫu. Nếu bài yêu cầu một cách sắp xếp thỏa điều kiện, hàm kiểm tra chỉ cần duyệt và xác nhận mọi điều kiện được thỏa. Nếu bài yêu cầu một lời giải tối ưu, hãy kiểm hai điều: kết quả hợp lệ, và giá trị của nó bằng giá trị mà lời giải vét cạn tìm được — so *giá trị* chứ không so cấu hình.

7. Trong công việc thật, quy trình này thay đổi thế nào?

| Cùng nguyên tắc, khác quy mô. Lời giải vét cạn trở thành một mô hình tham chiếu đơn giản hoặc một phiên bản cũ đã tin cậy. Bộ sinh dữ liệu trở thành sinh dữ liệu có ràng buộc theo miền nghiệp vụ. Bước thu nhỏ trở thành việc rút gọn một ca lỗi từ hệ thống thật thành một ca tái lập được. Và có

thêm hai thứ mà thi đấu không có: kiểm thử hồi quy — mỗi lỗi đã sửa trở thành một ca kiểm thử vĩnh viễn — và ghi nhật ký đủ để tái lập sự cố từ hiện trường.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Viết khung stress test dừng lại được cho thư viện cá nhân	Bài tập công cụ	Gồm bộ sinh, vòng so sánh, và hàm thu nhỏ; đây là bài tập có lợi ích lâu dài nhất chương.
● Viết lại tìm kiếm nhị phân và stress test nó	[2]	So với thư viện chuẩn trên hàng nghìn ca ngẫu nhiên, gồm mảng rỗng và mảng trùng lặp.
●● Dừng stress test để tìm phân ví dụ cho một tiêu chí tham lam sai	C/15	Mục tiêu kép: luyện công cụ và luyện thói quen không tin cảm giác.
●● Viết hàm thu nhỏ tự động cho phân ví dụ dạng mảng	Bài tập công cụ	Thử bỏ từng phần tử và giảm từng giá trị; giữ thay đổi nào vẫn còn lỗi.
●● Cố tình tạo một lỗi tràn số rồi tìm nó bằng quy trình ở mục 5	Bài tập luyện quy trình	Đi đúng thứ tự bốn bước; đo thời gian để thấy quy trình nhanh hơn sửa mò.
●● Chuyển một thuật toán đệ quy sâu sang bản lặp	Bài tập cài đặt	Duyệt sâu trên đồ thị 10^5 đỉnh hình đường thẳng; kiểm cả hai bản cho cùng kết quả.
●●● Viết bộ kiểm tra tính hợp lệ cho một bài có nhiều đáp án đúng	Bài tập kiểm thử	Ví dụ: sắp xếp tô-pô, ghép đôi, tô màu; kiểm mọi ràng buộc của đề.
●●● Kiểm một hàm trộn ngẫu nhiên bằng thống kê	Bài tập kiểm thử	Đếm tần suất mọi hoán vị với $n = 4$ trên 10^6 lần chạy; phát hiện lỗi lệch phân phối nếu có.
●●● Dừng git bisect để tìm thay đổi làm hỏng một bài trong kho mã của bạn	Bài tập công cụ	Trải nghiệm trực tiếp tìm kiếm nhị phân trên trục thời gian.
●●● Lập danh sách ca biên riêng cho ba dạng bài bạn hay làm	Bài tập tự tổ chức	Ví dụ: bài mảng, bài đồ thị, bài chuỗi; biến nó thành bảng kiểm trước khi nộp.

Kết chương và đọc tiếp

Chương này thay việc sửa mò bằng một quy trình. Ba điều đáng mang theo: thứ tự đúng khi gỡ lỗi — ca biên, stress test, thu nhỏ, rồi mới đọc code; ý thức rằng máy tìm phân ví dụ giỏi hơn người còn người hiểu phân ví dụ giỏi hơn máy, nên hãy phân công theo đúng thể mạnh; và việc chọn phép thu hẹp phù hợp trên ba trục không gian, thời gian và dữ liệu.

Chương tiếp theo mở rộng phạm vi: từ “chương trình chạy đúng” sang “chương trình chạy đủ nhanh trong một hệ thống thật”. Ở đó, các kết luận của phần A về độ phức tạp tiệm cận gặp thực tế của hằng số, bộ nhớ và tính bảo trì — và bạn sẽ thấy rằng thuật toán tốt nhất trên giấy thường không phải thuật toán được chọn (E/26).

Nguồn

- [1] A. Zeller and R. Hildebrandt, “Simplifying and Isolating Failure-Inducing Input,” *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [2] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.